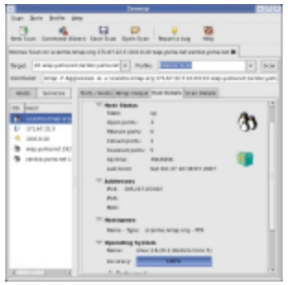
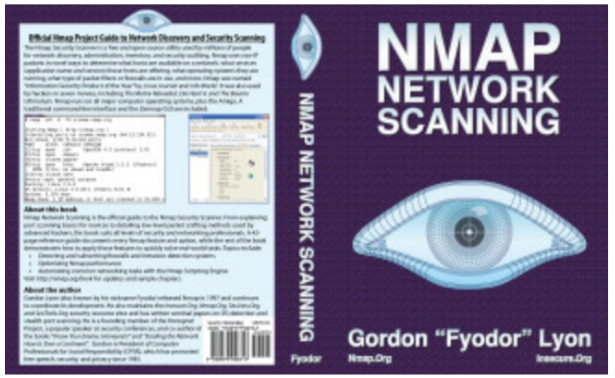
Nmap

https://nmap.org/book



Nmap Network Scanning



Nmap Network Scanning is the official guide to the Nmap Security Scanner, a free and open source utility used by millions of people for network discovery, administration, and security auditing. From explaining port scanning basics for novices to detailing low-level packet crafting methods used by advanced hackers, this book suits all levels of security and networking professionals. A 42-page reference guide documents every Nmap feature and option, while the rest of the book demonstrates how to apply those features to quickly solve real-world tasks. Examples and diagrams show actual communication on the wire.

Topics include subverting firewalls and intrusion detection systems, optimizing Nmap performance, and automating common networking tasks with the Nmap Scripting Engine. Hints and instructions are provided for common uses such as taking network inventory, penetration testing, detecting rogue wireless access points, and quashing network worm outbreaks. Nmap runs on Windows, Linux, and Mac OS X.

Nmap's original author, Gordon “Fyodor” Lyon, wrote this book to share everything he has learned about network scanning during more than a decade of Nmap development. It was briefly the #1 selling computer book on Amazon (screenshot). The book is in English, though several translations are in the works.

Key facts: The ISBN is 978-0-9799587-1-7 (ISBN-10 is 0-9799587-1-7) and suggested retail prices are $49.95 in the U.S., £34.95 in the U.K., and €39.95 in Europe. Like most books, it costs less online (as little as $32.97 - see purchasing options). It is 468 pages long. The official release date was January 1, 2009, though Amazon managed to beat that by a couple weeks.

About half of the content is available in the free online edition. Chapters exclusive to the print edition include “Detecting and Subverting Firewalls and Intrusion Detection Systems”, “Optimizing Nmap Performance”, “Port Scanning Te[Screenshot]gorithms”, “Host Discovery (Ping Scanning)”, and more. The solution selections which provide detailed instructions

Introduction to Nmap

- Nmap ("Network Mapper") is a free and open source utility for network discovery and security auditing.
- Many systems and network administrators also find it useful for tasks such as network inventory, managing service upgrade schedules, and monitoring host or service uptime.
- Nmap uses raw IP packets in novel ways to determine what hosts are available on the network, what services (application name and version) those hosts are offering, what operating systems (and OS versions) they are running, what type of packet filters/firewalls are in use, and dozens of other characteristics.
- It was designed to *rapidly scan large networks,* but works fine against single hosts.
- Nmap runs on all major computer operating systems, and official binary packages are available for Linux, Windows, and Mac OS X.
- In addition to the classic command-line Nmap executable,
  - the Nmap suite includes an advanced GUI and results viewer (Zenmap),
  - a flexible data transfer, redirection, and debugging tool (Ncat),
  - a utility for comparing scan results (Ndiff), and
  - a packet generation and response analysis tool (Nping).

Nmap is ...

- **Flexible**: Supports dozens of advanced techniques for mapping out networks filled with IP filters, firewalls, routers, and other obstacles. This includes many port scanning mechanisms (both TCP & UDP), OS detection, version detection, ping sweeps, and more. See the documentation page.
- **Powerful**: Nmap has been used to scan huge networks of literally hundreds of thousands of machines.
- **Portable**: Most operating systems are supported, including Linux, Microsoft Windows, FreeBSD, OpenBSD, Solaris, IRIX, Mac OS X, HP-UX, NetBSD, Sun OS, Amiga, and more.
- **Easy**: While Nmap offers a rich set of advanced features for power users, you can start out as simply as nmap -v -A *targethost*
- **Free**: The primary goals of the Nmap Project is to help make the Internet a little more secure and to provide administrators/auditors/hackers with an advanced tool for exploring their networks. Nmap is available for free download, and also comes with full source code that you may modify and redistribute under the terms of the license.
- **Well Documented**: Significant effort has been put into comprehensive and up-to-date man pages, whitepapers, tutorials, and even a whole book!
- **Supported**: While Nmap comes with no warranty, it is well supported by a vibrant community of developers and users. Most of this interaction occurs on the Nmap mailing lists. Most bug reports and questions should be sent to the nmap-dev list, but only after you read the guidelines. We recommend that all users subscribe to the low-traffic nmap-hackers announcement list. You can also find Nmap on Facebook and Twitter. For real-time chat, join the #nmap channel on Freenode or EFNet.
- **Acclaimed**: Nmap has won numerous awards, including "Information Security Product of the Year" by Linux Journal, Info World and Codetalker Digest. It has been featured in hundreds of magazine articles, several movies, dozens of books, and one comic book series. Visit the press page for further details.
- **Popular**: Thousands of people download Nmap every day, and it is included with many operating systems (Redhat Linux, Debian Linux, Gentoo, FreeBSD, OpenBSD, etc). It is among the top ten (out of 30,000) programs at the Freshmeat.Net repository. This is important because it lends Nmap its vibrant development and user support communities.

How to use Nmap

- "**nmap –h**" for help and "man nmap" for the manual pages on nmap.
- From the help, you can see the switches available and structure of the command, for example:
- **-sS** allows a SYN scan
- **-sU** for a UDP scan
- **-O** for operating system detection
- -**sV** for versions of the services

ICMP Network Scanning

On first connection to a target network in a black box assignment, the first objective is to obtain a "map" of the network structure" i.e. we want to see which IP addresses contain active hosts, and which do not.

One way to do this is by using Nmap to perform a so called "*ping sweep*". Nmap sends an ICMP packet to each possible IP address for the specified network. When it receives a response, it marks the IP address that responded as being alive.

To perform a ping sweep, we use the **-sn** switch in conjunction with IP ranges

- The **-sn** switch tells Nmap not to scan any ports, forcing it to rely purely on ICMP packets (or ARP requests on a local network) to identify targets.
- The IP range can be specified with either a hypen (-) or CIDR notation.
- For example, we could scan the 192.168.0.x network using:

  nmap -sn 192.168.0.1-254 i.e from host 1 to host 254

  or

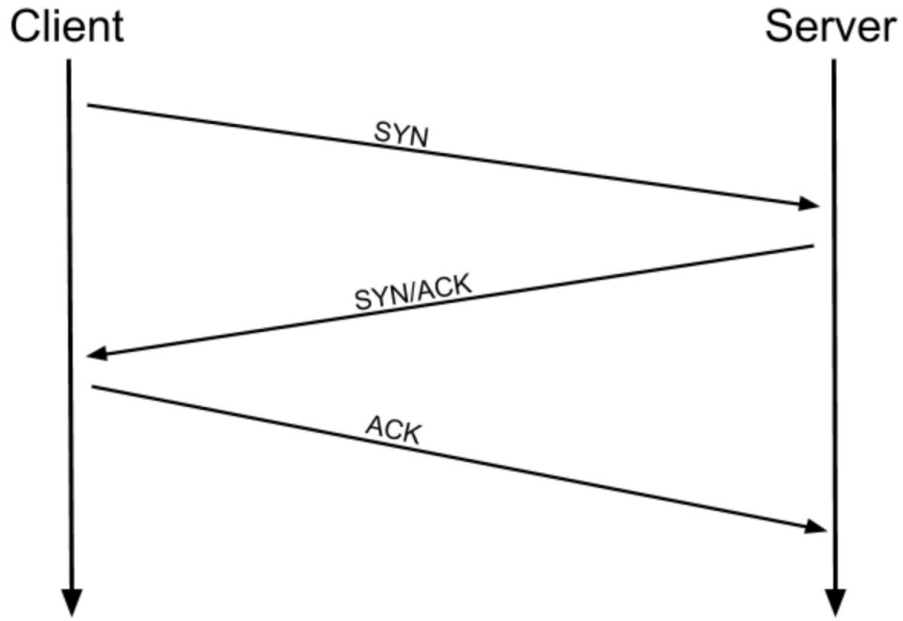
  nmap -sn 192.168.0.0/24 i.e all hosts in subnet

TCP Scans

A **TCP Connect scan** works by performing the three-way handshake (RFC 793) with each target port in turn trying to connect to each specified TCP port, and determines whether the service is open by the response it receives.

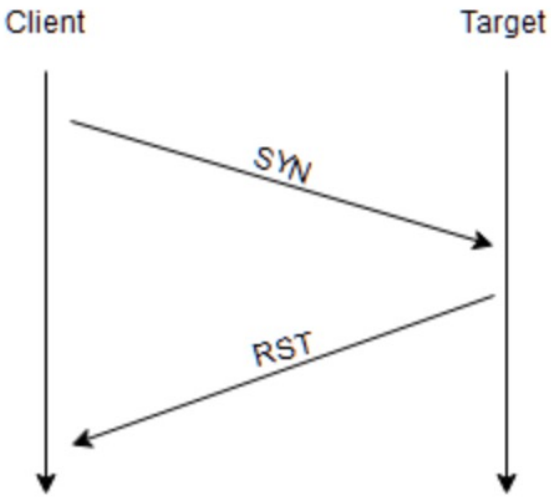
The three-way handshake consists of three stages.

1. First the client (our attacking machine) sends a TCP request to the target server with the SYN flag set.
2. The server then acknowledges this packet with a TCP response containing the SYN flag, as well as the ACK flag.
3. Finally, our client completes the handshake by sending a TCP request with the ACK flag set.



Source	Destination	Protocol	Length	Info
192.168.1.142	192.168.1.141	TCP	74	60516 → 80 [SYN] Seq=0 Win=64240 Len=0 MS
192.168.1.141	192.168.1.142	TCP	66	80 → 60516 [SYN, ACK] Seq=0 Ack=1 Win=655
192.168.1.142	192.168.1.141	TCP	54	60516 → 80 [ACK] Seq=1 Ack=1 Win=64256 Le

If Nmap sends a TCP request with the *SYN* flag set to a ***closed port***, the target server will respond with a TCP packet with the *RST* (Reset) flag set. Nmap records this as the port is closed.



What if the port is open, but hidden behind a firewall?

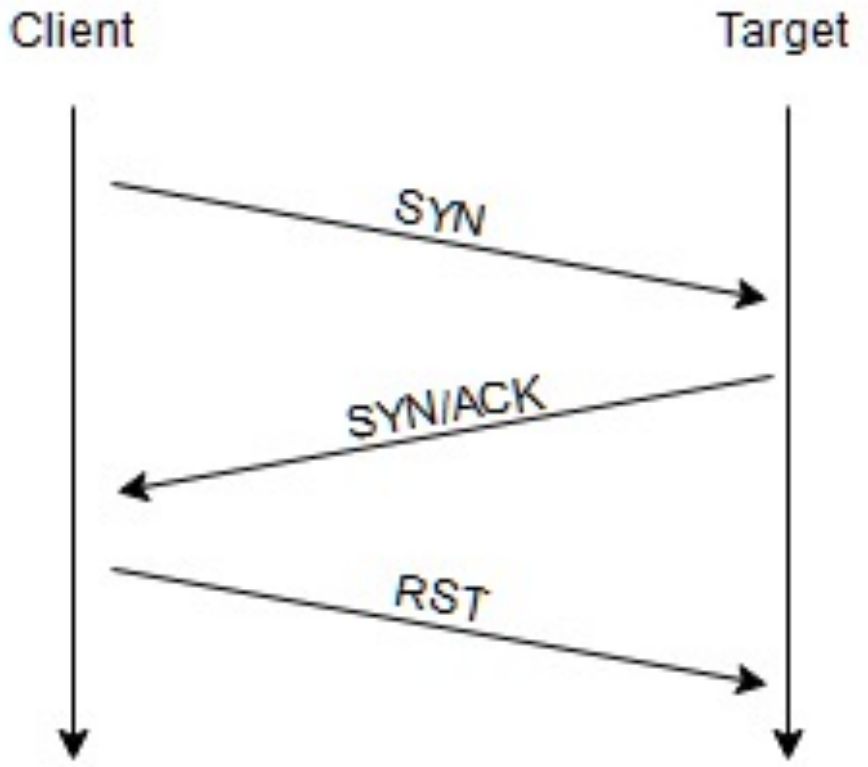
Many firewalls are configured to simply **drop** incoming packets. *Nmap sends a TCP SYN request, and receives nothing back.* This indicates that the port is being protected by a firewall and thus the port is considered to be *filtered*.

SYN Scans

SYN scans (**-sS**) are used to scan the TCP port-range of a target but works slightly differently from the TCP scan.

SYN scans are sometimes referred to as "*Half-open*" scans, or *"Stealth"* scans.

Where TCP scans perform a **full** three-way handshake with the target, SYN scans sends back a RST TCP packet after receiving a SYN/ACK from the server (this prevents the server from repeatedly trying to make the request).



T

No.	Time	Source	Destination	Protocol	Length	Info
39	8.389443540	192.168.1.142	192.168.1.238	TCP	58	53425 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
40	8.389900067	192.168.1.238	192.168.1.142	TCP	60	80 → 53425 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
41	8.389992786	192.168.1.142	192.168.1.238	TCP	54	53425 → 80 [RST] Seq=1 Win=0 Len=0

Advantages:

- It can be used to bypass older Intrusion Detection systems as they are looking out for a full three way handshake. it is for this reason that SYN scans are still frequently referred to as "stealth" scans.
- SYN scans are often not logged by applications listening on open ports, as standard practice is to log a connection once it's been fully established.
- SYN scans are significantly faster than a standard TCP Connect scan.

Disadvantages:

- They require sudo permissions in order to work correctly in Linux. This is because SYN scans require the ability to create raw packets (as opposed to the full TCP handshake), which is a privilege only the root user has by default.
- Unstable services are sometimes brought down by SYN scans, which could prove problematic if a client has provided a production environment for the test.

SYN scans are the default scans used by Nmap *if run with sudo permissions*.
If run **without** sudo permissions, Nmap defaults to the TCP Connect scan we saw in the previous task.

- SYN scan and TCP scan are identical for closed and filtered ports:
  - If a port is closed then the server responds with a RST TCP packet.
  - If the port is filtered by a firewall then the TCP SYN packet is either dropped, or spoofed with a TCP reset.
- The big difference is in how they handle *open* ports.

UDP Scans

- UDP connections are *stateless*. This means that, rather than initiating a connection with a back-and-forth "handshake", UDP connections rely on sending packets to a target port and *essentially hoping that they make it*.
- This makes UDP superb for connections which rely on *speed over quality* (e.g. video sharing), but the lack of acknowledgement makes UDP significantly more difficult (and much slower) to scan.
- The switch for an Nmap UDP scan is **-sU**
- When a packet is sent to an open UDP port, there should be no response. When this happens, Nmap refers to the port as being open|filtered.
- In other words, it suspects that the port is open, but it could be firewalled.
- If it gets a UDP response (which is very unusual), then the port is marked as *open*.
- More commonly there is no response, in which case the request is sent a second time as a double-check.
- If there is still no response then the port is marked *open|filtered* and Nmap moves on.

UDP Scans

- When a packet is sent to a *closed* UDP port, the target should respond with an ICMP (ping) packet containing a message that the *port is unreachable*. This clearly identifies **closed ports**, which Nmap marks as such and moves on.

- Due to this difficulty in identifying whether a UDP port is actually open, UDP scans tend to be incredibly slow in comparison to the various TCP scans (in the region of 20 minutes to scan the first 1000 ports, with a good connection). For this reason it's usually good practice to run an Nmap scan with --top-ports <number> enabled.

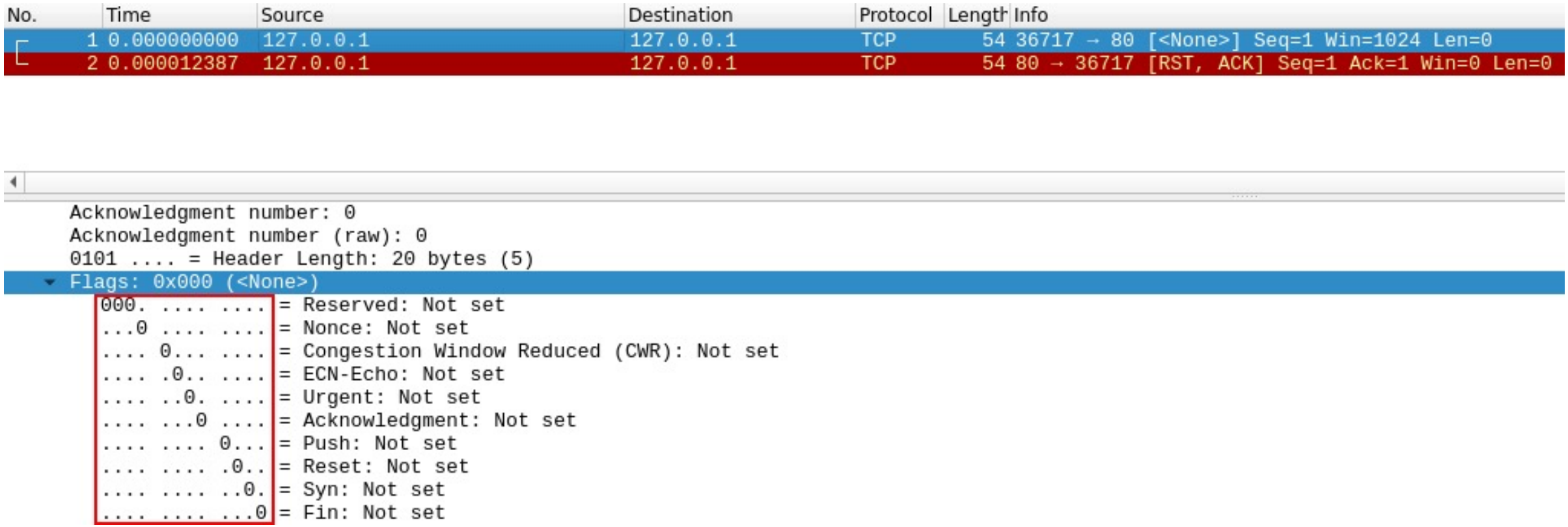
- When scanning UDP ports, Nmap usually sends completely empty requests -- just raw UDP packets. That said, for ports which are usually occupied by well-known services, it will instead send a protocol-specific payload which is more likely to elicit a response from which a more accurate result can be drawn.

NULL, FIN and Xmas TCP port scans

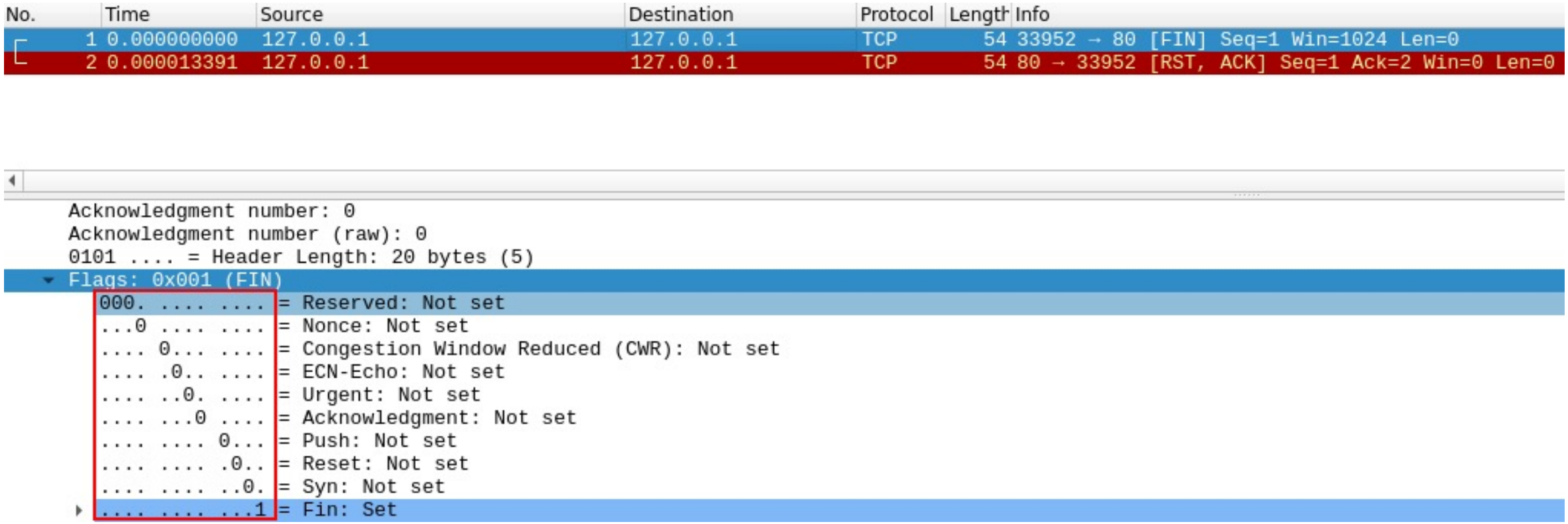
NULL, FIN and Xmas TCP port scans are less commonly used than any of the others covered here. All three are interlinked and are used primarily as they tend to be even stealthier, relatively speaking, than a SYN "stealth" scan.

NULL scans:

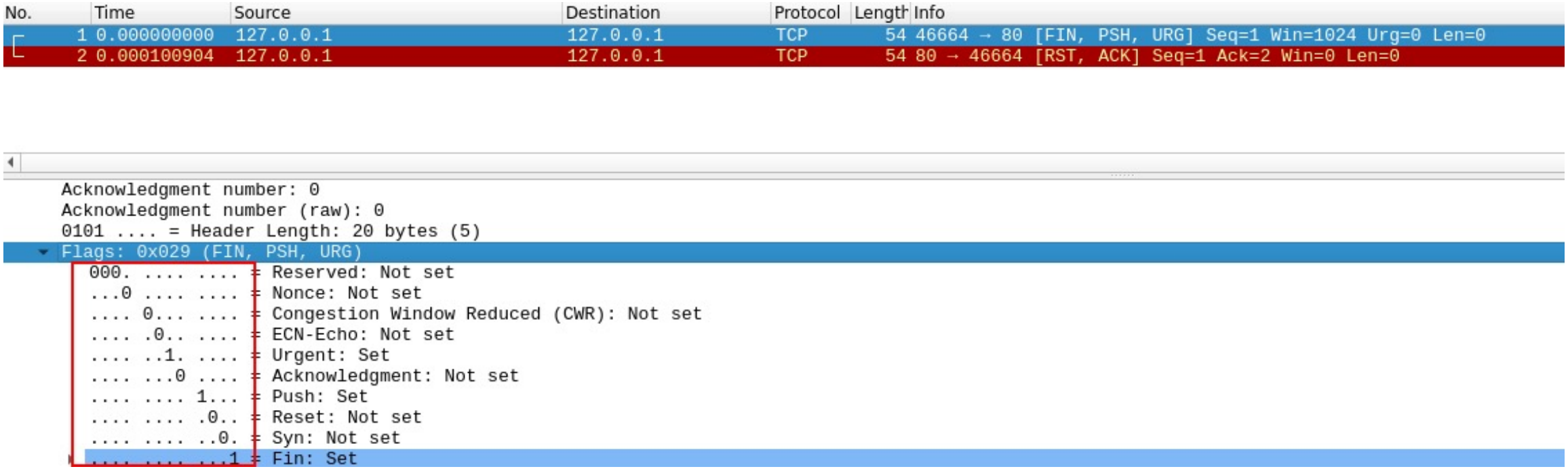
As the name suggests, NULL scans (**-sN**) are when the TCP request is sent with no flags set at all. As per the RFC, the target host should respond with a RST if the port is closed.



- **FIN scans** (-sF) work in an almost identical fashion; however, instead of sending a completely empty packet, a request is sent with the FIN flag (usually used to gracefully close an active connection). Once again, Nmap expects a RST if the port is closed.



- As with the other two scans in this class, **Xmas scans** (-sX) send a malformed TCP packet and expects a RST response for closed ports.
- It's referred to as an **xmas scan** as the flags that it sets (PSH, URG and FIN) give it the appearance of a *blinking christmas tree* when viewed as a packet capture in Wireshark.



- The expected response for *open ports* with these scans is also identical, and is very similar to that of a UDP scan. If the port is open then there is no response to the malformed packet.
- Unfortunately (as with open UDP ports), that is *also* an expected behaviour if the port is protected by a firewall, so NULL, FIN and Xmas scans will only ever identify ports as being *open|filtered*, *closed*, or *filtered*. If a port is identified as filtered with one of these scans then it is usually because the target has responded with an ICMP unreachable packet.
- It's also worth noting that while RFC 793 mandates that network hosts respond to malformed packets with a RST TCP packet for closed ports, and don't respond at all for open ports; *this is not always the case in practice*. In particular Microsoft Windows (and a lot of Cisco network devices) are known to respond with a RST to any malformed TCP packet -- regardless of whether the port is actually open or not. This results in all ports showing up as being closed.
- That said, the goal here is, of course, firewall evasion. Many firewalls are configured to drop incoming TCP packets to blocked ports which have the SYN flag set (thus blocking new connection initiation requests). By sending requests which do not contain the SYN flag, we effectively bypass this kind of firewall.
- Modern IDS solutions are savvy to these scan types, so don't rely on them to be 100% effective when dealing with modern systems.

Summary Table of Nmap Scanning

	Open	Closed	Filtered
TCP scan	SNY/ACK and rely with ACK	RST	Nothing
SYN scan	SYN/ACK and reply with RST	RST	Nothing
UDP	No response / Response (rare)	ICMP “Port Unreachable”	No Response
NULL	No response	RST	ICMP “Port Unreachable”
FIN	No response	RST	ICMP “Port Unreachable”
Xmas	No response	RST	ICMP “Port Unreachable”

Firewall Evasion

Windows host default firewall will block all ICMP packets. This presents a problem as Nmap uses ICMP by default will register a host blocking ICMP packets as dead and not bother scanning it at all.

- Nmap provides an option: **-Pn**, which tells Nmap to not bother pinging the host before scanning it. This treats the target host(s) as being alive, effectively bypassing the ICMP block but potentially taking a very long time to complete the scan (if the host really is dead, Nmap will still be checking and double checking every specified port).
- It's worth noting that if you're already directly on the local network, Nmap can also use ARP requests to determine host activity.

There are a variety of other switches which Nmap considers useful for firewall evasion.

- -f:- Used to fragment the packets (i.e. split them into smaller pieces) making it less likely that the packets will be detected by a firewall or IDS.
- An alternative to -f, but providing more control over the size of the packets: --mtu <number>, accepts a maximum transmission unit size to use for the packets sent. This *must* be a multiple of 8.
- --scan-delay <time>ms:- used to add a delay between packets sent. This is very useful if the network is unstable, but also for evading any time-based firewall/IDS triggers which may be in place.
- --badsum:- this is used to generate in invalid checksum for packets. Any real TCP/IP stack would drop this packet, however, firewalls may potentially respond automatically, without bothering to check the checksum of the packet. As such, this switch can be used to determine the presence of a firewall/IDS.

Nmap Scripting Engine (NSE)

- The **Nmap Scripting Engine (NSE)** is an incredibly powerful addition to Nmap, extending its functionality quite considerably. NSE Scripts are written in the *Lua* programming language, and can be used to do a variety of things: from scanning for vulnerabilities, to automating exploits for them. The NSE is particularly useful for reconnaissance, however, it is well worth bearing in mind how extensive the script library is.

Some useful categories include:

- **safe:**- Won't affect the target
- **intrusive**:- Not safe: likely to affect the target
- **vuln**:- Scan for vulnerabilities
- **exploit**:- Attempt to exploit a vulnerability
- **auth**:- Attempt to bypass authentication for running services (e.g. Log into an FTP server anonymously)
- **brute**:- Attempt to bruteforce credentials for running services
- **discovery**:- Attempt to query running services for further information about the network (e.g. query an SNMP server).

How to *find* scripts.

- We have two options for this, which should ideally be used in conjunction with each other.
  1. The first is the page on the Nmap website which contains a list of all official scripts.
  2. The second is the local storage on your attacking machine. Nmap stores its scripts on Linux at **/usr/share/nmap/scripts**. All of the NSE scripts are stored in this directory by default -- this is where Nmap looks for scripts when you specify them.
- There are two ways to search for installed scripts.
  1. One is by using the **/usr/share/nmap/scripts/script.db** file. Despite the extension, this isn't actually a database so much as a formatted text file containing filenames and categories for each available script.

```
muri@augury:/usr/share/nmap/scripts$ file script.db
script.db: ASCII text
muri@augury:/usr/share/nmap/scripts$ head script.db
Entry { filename = "acarsd-info.nse", categories = { "discovery", "safe", } }
Entry { filename = "address-info.nse", categories = { "default", "safe", } }
Entry { filename = "afp-brute.nse", categories = { "brute", "intrusive", } }
Entry { filename = "afp-ls.nse", categories = { "discovery", "safe", } }
Entry { filename = "afp-path-vuln.nse", categories = { "exploit", "intrusive", "vuln", } }
Entry { filename = "afp-serverinfo.nse", categories = { "default", "discovery", "safe", } }
Entry { filename = "afp-showmount.nse", categories = { "discovery", "safe", } }
Entry { filename = "ajp-auth.nse", categories = { "auth", "default", "safe", } }
Entry { filename = "ajp-brute.nse", categories = { "brute", "intrusive", } }
Entry { filename = "ajp-headers.nse", categories = { "discovery", "safe", } }
```

- Nmap uses this file to keep track of (and utilise) scripts for the scripting engine; however, we can also ***grep*** through it to look for scripts. For example: grep "ftp" /usr/share/nmap/scripts/script.db.

```
muri@augury:/usr/share/nmap/scripts$ grep "ftp" /usr/share/nmap/scripts/script.db
Entry { filename = "ftp-anon.nse", categories = { "auth", "default", "safe", } }
Entry { filename = "ftp-bounce.nse", categories = { "default", "safe", } }
Entry { filename = "ftp-brute.nse", categories = { "brute", "intrusive", } }
Entry { filename = "ftp-libopie.nse", categories = { "intrusive", "vuln", } }
Entry { filename = "ftp-proftpd-backdoor.nse", categories = { "exploit", "intrusive", "malware", "vuln", } }
Entry { filename = "ftp-syst.nse", categories = { "default", "discovery", "safe", } }
Entry { filename = "ftp-vsftpd-backdoor.nse", categories = { "exploit", "intrusive", "malware", "vuln", } }
Entry { filename = "ftp-vuln-cve2010-4221.nse", categories = { "intrusive", "vuln", } }
Entry { filename = "tftp-enum.nse", categories = { "discovery", "intrusive", } }
```

2. The second way to search for scripts is quite simply to use the ls command. For example, we could get the same results as in the previous screenshot by using. ls -l /usr/share/nmap/scripts/*ftp*:

```
muri@augury:/usr/share/nmap/scripts$ ls -l /usr/share/nmap/scripts/*ftp*
-rw-r--r-- 1 root root 4530 Oct 12 14:29 /usr/share/nmap/scripts/ftp-anon.nse
-rw-r--r-- 1 root root 3253 Oct 12 14:29 /usr/share/nmap/scripts/ftp-bounce.nse
-rw-r--r-- 1 root root 3108 Oct 12 14:29 /usr/share/nmap/scripts/ftp-brute.nse
-rw-r--r-- 1 root root 3272 Oct 12 14:29 /usr/share/nmap/scripts/ftp-libopie.nse
-rw-r--r-- 1 root root 3290 Oct 12 14:29 /usr/share/nmap/scripts/ftp-proftpd-backdoor.nse
-rw-r--r-- 1 root root 3768 Oct 12 14:29 /usr/share/nmap/scripts/ftp-syst.nse
-rw-r--r-- 1 root root 6021 Oct 12 14:29 /usr/share/nmap/scripts/ftp-vsftpd-backdoor.nse
-rw-r--r-- 1 root root 5923 Oct 12 14:29 /usr/share/nmap/scripts/ftp-vuln-cve2010-4221.nse
-rw-r--r-- 1 root root 5736 Oct 12 14:29 /usr/share/nmap/scripts/tftp-enum.nse
```

- The same techniques can also be used to search for categories of script. For example: grep "safe" /usr/share/nmap/scripts/script.db

```
muri@augury:/usr/share/nmap/scripts$ grep "safe" /usr/share/nmap/scripts/script.db
Entry { filename = "acarsd-info.nse", categories = { "discovery", "safe", } }
Entry { filename = "address-info.nse", categories = { "default", "safe", } }
Entry { filename = "afp-ls.nse", categories = { "discovery", "safe", } }
Entry { filename = "afp-serverinfo.nse", categories = { "default", "discovery", "safe", } }
Entry { filename = "afp-showmount.nse", categories = { "discovery", "safe", } }
Entry { filename = "ajp-auth.nse", categories = { "auth", "default", "safe", } }
Entry { filename = "ajp-headers.nse", categories = { "discovery", "safe", } }
Entry { filename = "ajp-methods.nse", categories = { "default", "safe", } }
Entry { filename = "ajp-request.nse", categories = { "discovery", "safe", } }
Entry { filename = "allseeingeye-info.nse", categories = { "discovery", "safe", "version", } }
```

Installing New Scripts

We mentioned previously that the Nmap website contains a list of scripts, so, what happens if one of these is missing in the scripts directory locally? A standard sudo apt update && sudo apt install nmap should fix this; however, it's also possible to install the scripts manually by downloading the script from Nmap (sudo wget -O /usr/share/nmap/scripts/<script-name>.nse https://svn.nmap.org/nmap/scripts/<script-name>.nse). This must then be followed up with nmap --script-updatedb, which updates the script.db file to contain the newly downloaded script.
It's worth noting that you would require the same "updatedb" command if you were to make your own NSE script and add it into Nmap -- a more than manageable task with some basic knowledge of Lua!

- The --script switch is for activating NSE scripts from the **vuln category** using --script=vuln. Other categories work in exactly the same way. If the command --script=safe is run, then any applicable safe scripts will be run against the target (Note: only scripts which target an active service will be activated).
- To run a specific script, we would use --script=<script-name> , e.g. --script=http-fileupload-exploiter.
- Multiple scripts can be run simultaneously in this fashion by separating them by a comma. For example: --script=smb-enum-users,smb-enum-shares.
- Some scripts require arguments (for example, credentials, if they're exploiting an authenticated vulnerability). These can be given with the --script-args Nmap switch. An example of this would be with the http-put script (used to upload files using the PUT method). This takes two arguments: the URL to upload the file to, and the file's location on disk. For example:
- nmap -p 80 --script http-put --script-args http-put.url='/dav/shell.php',http-put.file='./shell.php'
- Note that the arguments are separated by commas, and connected to the corresponding script with periods (i.e. <script-name>.<argument>).
- Nmap scripts come with built-in help menus, which can be accessed using nmap --script-help <script-name>. This tends not to be as extensive as in the link given above, however, it can still be useful when working locally.